Detección de objetos

La detección de objetos es un área dentro de la **visión por computador** (así como lo es la clasificación de imágenes que hemos visto detalladamente en el cuadernillo `502`).

Esta técnica es más complicada que la anterior, dado que no solo buscamos determinar una etiqueta para una imagen, sino determinar en qué posición de dicha imagen existe un determinado objeto.

Esta técnica es la técnica por excelencia a la hora de hablar de la Inteligencia Artificial en los medios de comunicación, son los típicos casos en los que a una imagen se le añade un recuadro verde que indica que dentro de ese recuadro *existe* un objeto.

Fíjate, por tanto, que estos *objetos* pueden ser también seres vivos, como personas o animales. En general, no buscamos determinar *dónde se encuentra el objeto*, sino *dónde se encuentran TODOS los objetos* que el modelo pueda determinar: es decir, en una misma imagen podemos encontrar varios recuadros (que, a partir de ahora, llamaremos **bboxes o *bounding boxes***).

Como esta técnica es una especie de *evolución* de la clasificación de imágenes, podemos aplicar todas las técnicas vistas en el cuadernillo `502` en este nuevo cuadernillo. **¡Pero cuidado!**: aplicar algunas técnicas podría no tener sentido o producir valores sin sentido. Por ejemplo, a la hora de aplicar aumento de datos, también tendremos que evaluar los cambios que sufren las ***bounding boxes***, por ejemplo, en un giro o en una perspectiva.

La métrica por excelencia en la detección de objetos

Fíjate en que la detección de objetos no es un problema de regresión ni de clasificación, por lo que las métricas que hemos visto hasta ahora no nos sirven: tenemos que definir otras.

En general, se suele usar la métrica IoU (*intersection over union*), que determina cómo de bueno es el rectángulo generado según la etiqueta aportada. Por cierto: no lo hemos mencionado, pero claramente las *etiquetas* que necesitan los datos para este tipo de problemas es una lista de las coordenadas en las que se encuentran las **bboxes**. Por simplicidad, en este cuadernillo solo usaremos la `valid_loss`.

Hay varias formas de determinar estas coordenadas, y cada forma de hacerlo da lugar a nuevas codificaciones y nomenclaturas. Las más comunes con `PASCAL VOC` y `COCO`, que usan formatos `xml` y `json` respectivamente.

Además, también tenemos distintas arquitecturas para entrenar modelos que realicen esta tarea, como `YOLO` (versiones hasta la 8) o `FCOS`, entre otras muchas.

Ten en cuenta que, dado que actualmente la **visión por computador** está en un avance constante, las estrategias que menciono aquí (aunque sean *relativamente* nuevas) podrían

quedar desfasadas. En ese sentido, no te centres en cómo se usan, sino en qué se busca encontrar conceptualmente para entrenar estos modelos.

Un modelo de detección de objetos

Hasta ahora hemos usado la librería de FastAI para entrenar nuestros modelos. Sin embargo, esta librería funciona especialmente mal para los modelos de tipo **detección de objetos**, ya que son modelos bicéfalos (reciben una imagen y sus anotaciones), generando una imagen anotada. Por eso, **NO** seguiremos el tutorial que nos sugieren (aunque te lo dejo por completitud) `FastAI`.

Para arreglar este problema, usaremos otra librería de *Deep Learning*: `HuggingFace`. La forma de trabajar es similar, aunque tiene ciertas diferencias.

Iremos poco a poco explicando todos los detalles, pero no dudes en preguntas todas las dudas que tengas. Como siempre.

Librerías necesarias y `token` de acceso

Importaremos las librerías:

- `datasets`. Para trabajar con los `datasets` de `HuggingFace` (que son ligeramente diferentes a los de `FastAI`, aunque conceptualmente es lo mismo: lugares de donde recoger los datos.
- `transformers`. Para trabajar con los `transformers` de `HuggingFace`. A efectos prácticos: sus modelos.
- `accelerate`. Para mejorar varios procesos computacionales.
- `timm`. Ya vista en clasificación de imágenes.
- `albumentations`. Ya vista en clasificación de imágenes.
- `torchmetrics`. Para definir varias métricas.
- `pycocotools`. Para ayudarnos con la nomenclatura de las anotaciones.

```
!pip install -q datasets transformers accelerate timm
!pip install -q -U albumentations torchmetrics pycocotools
```

Nos registramos con nuestro `token` de `HuggingFace` para poder entrenar nuestro modelo más adelante.

```
from huggingface_hub import notebook_login

notebook_login()
```

Finalmente, definimos algunos parámetros globales del cuadernillo:

- `MODEL_NAME`. El modelo con el que vamos a trabajar para hacer fine-tune. Podríamos usar cualquier otro del repositorio de `HuggingFace`.
- `IMAGE_SIZE`. El tamaño máximo de las imágenes con las que vamos a trabajar. Cuanto más grande sea el tamaño, más capacidad de cómputo necesitaremos.

```
MODEL_NAME = "microsoft/conditional-detr-resnet-50"
IMAGE_SIZE = 480
```

Recopilando y preprocesando los datos

En este problema, y para cambiar un poco de animales, vamos a tomar imágenes de canguros y vamos a intentar detectar en qué parte de la imagen está dicho canguro.

Para ello, vamos a descargarnos un *dataset* de canguros.

```
!git clone https://github.com/experiencor/kangaroo
```

```
fatal: destination path 'kangaroo' already exists and is not an empty directory.
```

Puedes ver, en el explorador de archivos, que ahora tienes un *dataset* llamado `kangaroo` que tiene dos carpetas: `annots` y `images`.

Fíjate (esto es muy importante siempre en los problemas de detección) en cuál es el formato de las etiquetas. En este caso, el formato es `xml`, por lo que las etiquetas están en nomenclatura `PASCAL VOC`.

Como queremos entrenar la arquitectura DETR, necesitamos que las notaciones estén en formato COCO. Para poder transformar las notaciones, podríamos crear un *script* para modificarlo... pero existen otras herramientas que son muy útiles y es muy recomendado que las conozcas. En este caso, hablaremos de roboflow.

En el aula virtual puedes ver el proceso de carga de datos que necesitaremos para este cuadernillo. Para poder pasar los datos a `roboflow`, comprimimos y descargamos el conjunto de datos.

```
%%capture
!zip -r kangaroo.zip kangaroo
```

A la izquierda, descarga el conjunto de datos y realiza todos los pasos de `roboflow`.

Después, sube el `zip` generado a `Google Colab` y continúa.

Descomprimimos el conjunto de datos y creamos las carpetas que usaremos para el entrenamiento. Recuerda que, como en el cuadernillo `502`, también creamos variables para guardar las rutas de estas carpetas.

```
%%capture
!unzip 503-kangaroo.v2i.coco.zip
```

```
!mkdir dataset
!mv test dataset
!mv train dataset
```

Creación del `dataset`

Ahora tenemos que construir un nuevo `dataset` del tipo `HuggingFace` que recoja nuestros datos.

Lo primero que necesitamos es un generador que construya los objetos tal y como los espera recibir `HuggingFace`.

La librería espera recibir un diccionario que defina la información para cada objeto.

```
import json
import os
from PIL import Image

def gen():
    data_path = "/content/dataset/train"
    with open(os.path.join(data_path, "_annotations.coco.json"), "r") as f:
        annotations = json.load(f)

    for image in annotations["images"]:
        image_data = dict()
        image_data["image_id"] = image["id"] # un identificador único
        image_data["image"] = Image.open(os.path.join(data_path, image["file_name"]))
        image_data["width"] = image["width"] # la anchura
        image_data["height"] = image["height"] # la altura

        image_data["objects"] = dict() # las anotaciones
        related_annotations = [ann for ann in annotations["annotations"] if ann["image
        image_data["objects"]["id"] = [ann["id"] for ann in related_annotations] # los
        image_data["objects"]["area"] = [ann["area"] for ann in related_annotations] #
        image_data["objects"]["bbox"] = [ann["bbox"] for ann in related_annotations]
        image_data["objects"]["category"] = [ann["category_id"] - 1 for ann in relatec

        yield image_data # los generadores no devuelven objetos, sino contextos (yielc
```

Ahora podemos cargar nuestros datos usando el generador que acabamos definir.

```
from datasets import Dataset

dataset = Dataset.from_generator(gen)
print(dataset)
```

```
Generating train split:        162/0 [00:00<00:00,   6.02 examples/s]
Dataset({
    features: ['image_id', 'image', 'width', 'height', 'objects'],
    num_rows: 162
})
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Nuestro dataset es un solo conjunto con todos los datos. Vamos a crear los conjuntos de validación y testeo.

```
dataset = dataset.train_test_split(0.2, seed = 33)
split = dataset["train"].train_test_split(0.15, seed = 33)
dataset["train"] = split["train"]
dataset["validation"] = split["test"]

dataset
```

```
DatasetDict({
    train: Dataset({
        features: ['image_id', 'image', 'width', 'height', 'objects'],
        num_rows: 109
    })
    test: Dataset({
        features: ['image_id', 'image', 'width', 'height', 'objects'],
        num_rows: 33
    })
    validation: Dataset({
        features: ['image_id', 'image', 'width', 'height', 'objects'],
        num_rows: 20
    })
})
```

```
dataset["train"][10] # un ejemplo de cómo se guarda la información
```

```
{'image_id': 21,
 'image': <PIL.JpegImagePlugin.JpegImageFile image mode=RGB size=600x394>,
 'width': 600,
 'height': 394,
 'objects': {'area': [30751],
  'bbox': [[338, 92, 191, 161]],
  'category': [0],
  'id': [30]}}
```

Como siempre hemos dicho: mostramos un ejemplo para ver que todo se ha cargado correctamente.

```
import numpy as np
from PIL import ImageDraw

image = dataset["train"][30]["image"]
annotations = dataset["train"][30]["objects"]
draw = ImageDraw.Draw(image)

categories = ["kangaroo"] # nuestras posibles clases

id2label = {index: x for index, x in enumerate(categories)}
label2id = {v: k for k, v in id2label.items()}

for i in range(len(annotations["id"])):
    box = annotations["bbox"][i]
    class_idx = annotations["category"][i]
```

```
    x, y, w, h = tuple(box)

    # Check if coordinates are normalized or not
    if max(box) > 1.0:
        # Coordinates are un-normalized, no need to re-scale them
        x1, y1 = int(x), int(y)
        x2, y2 = int(x + w), int(y + h)
    else:
        # Coordinates are normalized, re-scale them
        x1 = int(x * image.width)
        y1 = int(y * image.height)
        x2 = int((x + w) * image.width)
        y2 = int((y + h) * image.height)

    draw.rectangle((x, y, x + w, y + h), outline="red", width=1)
    draw.text((x, y), id2label[class_idx], fill="white")

image
```

Aumento de datos

Ahora vamos a definir cómo queremos aumentar nuestros datos en tiempo de procesamiento.

Vamos a solicitar los siguientes cambios:

- `do_resize`. Escala los datos a las dimensiones adecuadas para el modelo.
- `do_pad`. En el caso de que un aumento reduzca el tamaño de una imagen, añade un *padding*.

```
from transformers import AutoImageProcessor

MAX_SIZE = IMAGE_SIZE

image_processor = AutoImageProcessor.from_pretrained(
    MODEL_NAME,
    do_resize = True,
    size = {"max_height": MAX_SIZE, "max_width": MAX_SIZE},
    do_pad = True,
    pad_size = {"height": MAX_SIZE, "width": MAX_SIZE},
)
```

```
preprocessor_config.json: 100%                                    301/301 [00:00<00:00, 31.7kB/s]
The image processor of type `ConditionalDetrImageProcessor` is now loaded as a fast prc
```

Finalmente, definimos los aumentos de datos tradicionales, como vimos en `502`.

Hay un parámetro muy importante llamado `min_visibility` que nos permite eliminar las `bboxes` que, después del procesamiento, se hayan quedado demasiado pequeñas (se hayan cortado, reducido, etcétera).

```
import albumentations as A

train_augment_and_transform = A.Compose(
    [
        A.Perspective(p=0.1, fit_output=True), # modifica la perspectiva con un 10% de
        A.HorizontalFlip(p=0.5), # haz un giro horizontal (50%)
        A.RandomBrightnessContrast(p=0.5), # añade contraste de brillo aleatorio (50%)
        A.HueSaturationValue(p=0.1), # añade saturación (10%)
    ],
    bbox_params=A.BboxParams(format="coco", label_fields=["category"], min_visibility=
)

validation_transform = A.Compose(
    [A.NoOp()], # no hagas nada
    bbox_params=A.BboxParams(format="coco", label_fields=["category"]),
)
```

Ahora vamos a definir cómo se realizan estos cambios en tiempo de entrenamiento y validación. Para ello, necesitamos definir dos funciones auxiliares.

- `format_image_annotations_as_coco`. Hace los cambios pertinentes para volver a la notación COCO (simplemente añade un paráemtro llamado `iscrowd` a los resultados obtenidos de la transformación).
- `augment_and_transform_batch`. Realiza los cambios que hemos definido anteriormente (el aumento de datos). A todos los efectos prácticos, es como la clase

`TransformPipeline` que hemos definido previamente en otros cuadernillos.

```
def format_image_annotations_as_coco(image_id, categories, areas, bboxes):
    """Format one set of image annotations to the COCO format

    Args:
        image_id (str): image id. e.g. "0001"
        categories (List[int]): list of categories/class labels corresponding to provi
        areas (List[float]): list of corresponding areas to provided bounding boxes
        bboxes (List[Tuple[float]]): list of bounding boxes provided in COCO format
            ([center_x, center_y, width, height] in absolute coordinates)

    Returns:
        dict: {
            "image_id": image id,
            "annotations": list of formatted annotations
        }
    """
    annotations = []
    for category, area, bbox in zip(categories, areas, bboxes):
        formatted_annotation = {
            "image_id": image_id,
            "category_id": category,
            "iscrowd": 0,
            "area": area,
            "bbox": list(bbox),
        }
        annotations.append(formatted_annotation)

    return {
        "image_id": image_id,
        "annotations": annotations,
    }
```

```
def augment_and_transform_batch(examples, transform, image_processor, return_pixel_mas
    """Apply augmentations and format annotations in COCO format for object detection

    images = []
    annotations = []
    for image_id, image, objects in zip(examples["image_id"], examples["image"], examp
        image = np.array(image.convert("RGB"))

        # Normalize bounding boxes to [0, 1]
        bboxes = objects["bbox"]
        bboxes = [[x / image.shape[1], y / image.shape[0], w / image.shape[1], h / ima

        # apply augmentations
        output = transform(image=image, bboxes=bboxes, category=objects["category"])
        images.append(output["image"])

        # Clip bounding boxes to [0, 1] after augmentation
        output["bboxes"] = [[np.clip(x, 0, 1), np.clip(y, 0, 1), np.clip(w, 0, 1), np.

        # format annotations in COCO format
        formatted_annotations = format_image_annotations_as_coco(
            image_id, output["category"], objects["area"], output["bboxes"]
```

```
        )
        annotations.append(formatted_annotations)

    # Apply the image processor transformations: resizing, rescaling, normalization
    result = image_processor(images=images, annotations=annotations, return_tensors="p

    if not return_pixel_mask:
        result.pop("pixel_mask", None)

    return result
```

Finalmente, apoyándonos en las funciones auxiliares, definimos las transformaciones para cada conjunto de datos de nuestro *dataset*.

```
from functools import partial # esta función me permite definir funciones que reciban

train_transform_batch = partial(augment_and_transform_batch, transform=train_augment_a
validation_transform_batch = partial(augment_and_transform_batch, transform=validatior

dataset["train"] = dataset["train"].with_transform(train_transform_batch)
dataset["validation"] = dataset["validation"].with_transform(validation_transform_batc
dataset["test"] = dataset["test"].with_transform(validation_transform_batch)
```

Podemos ver cómo ha quedado la misma imagen que habíamos mostrado como ejemplo, ahora se ve así (ya está lista para ser utilizada).

```
dataset["train"][30]
```

```
{'pixel_values': tensor([[[-1.1760, -1.1932, -1.1589,  ..., -0.8507, -0.9192,
-0.9877],
          [-1.1075, -1.0562, -1.0733,  ..., -0.7308, -0.9877, -0.9534],
          [-0.8507, -0.8335, -1.0219,  ..., -0.6623, -0.7650, -0.7137],
          ...,
          [ 0.0000,  0.0000,  0.0000,  ...,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  ...,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  ...,  0.0000,  0.0000,  0.0000]],

         [[-0.6001, -0.6001, -0.5651,  ..., -0.4076, -0.4951, -0.5651],
          [-0.5126, -0.4601, -0.4951,  ..., -0.2850, -0.5651, -0.5301],
          [-0.2675, -0.2500, -0.4601,  ..., -0.2150, -0.3200, -0.3025],
          ...,
          [ 0.0000,  0.0000,  0.0000,  ...,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  ...,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  ...,  0.0000,  0.0000,  0.0000]],

         [[-1.8044, -1.8044, -1.8044,  ..., -1.6650, -1.6824, -1.6999],
          [-1.8044, -1.8044, -1.8044,  ..., -1.6127, -1.7522, -1.7347],
          [-1.8044, -1.8044, -1.8044,  ..., -1.5779, -1.6650, -1.6476],
          ...,
          [ 0.0000,  0.0000,  0.0000,  ...,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  ...,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  ...,  0.0000,  0.0000,  0.0000]]]),
 'labels': {'size': tensor([480, 480]), 'image_id': tensor([137]), 'class_labels':
tensor([0, 0]), 'boxes': tensor([[7.2812e-04, 5.0833e-04, 4.0312e-04, 5.9167e-04],
         [6.4375e-04, 6.3958e-04, 8.7500e-05, 1.4583e-04]]), 'area':
tensor([26377.9219,  1411.2001]), 'iscrowd': tensor([0, 0]), 'orig_size': tensor([600,
800])}}
```

Entrenamiento (preparación)

Para realizar nuestro entrenamiento, necesitamos definir algunas funciones adicionales para trabajar con los datos. Las explicamos de forma general:

Primero, necesitamos una función que nos permita "agrupar" los datos de un *batch*. Recuerda que, en el entrenamiento, varias imágenes se envían a la vez al modelo. Estas imágenes se agrupan en la misma matriz para ser procesadas de forma paralela.

```
import torch

def collate_fn(batch):
    data = {}
    data["pixel_values"] = torch.stack([x["pixel_values"] for x in batch])
    data["labels"] = [x["labels"] for x in batch]
    if "pixel_mask" in batch[0]:
        data["pixel_mask"] = torch.stack([x["pixel_mask"] for x in batch])

    return data
```

Aunque estemos trabajando con COCO, necesitamos una función que convierta los resultados a PASCAL para poder utilizar métricas que ya estén definidas. Esta función hace esa conversión.

```
from transformers.image_transforms import center_to_corners_format

def convert_bbox_yolo_to_pascal(boxes, image_size):
    """
    Convert bounding boxes from YOLO format (x_center, y_center, width, height) in rar
    to Pascal VOC format (x_min, y_min, x_max, y_max) in absolute coordinates.

    Args:
        boxes (torch.Tensor): Bounding boxes in YOLO format
        image_size (Tuple[int, int]): Image size in format (height, width)

    Returns:
        torch.Tensor: Bounding boxes in Pascal VOC format (x_min, y_min, x_max, y_max)
    """
    # convert center to corners format
    boxes = center_to_corners_format(boxes)

    # convert to absolute coordinates
    height, width = image_size
    boxes = boxes * torch.tensor([[width, height, width, height]])

    return boxes
```

Entrenamiento (definición)

Por fin, podemos definir nuestro `modelo`.

```
from transformers import AutoModelForObjectDetection

model = AutoModelForObjectDetection.from_pretrained(
    MODEL_NAME, # nombre del modelo sobre el que hacer finetune
    id2label=id2label, # la función para pasar de ids a nombres
    label2id=label2id, # la función para pasar de nombres a ids
    ignore_mismatched_sizes=True, # ignoramos los tamaños incorrectos
)
```

```
config.json:          4.43k/?  [00:00<00:00,  373kB/s]

model.safetensors:  100%                                    174M/174M  [00:02<00:00,  127MB/s]

model.safetensors:  100%                                    102M/102M  [00:01<00:00,  21.6MB/s]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
```

Modelo creado, ahora necesitamos definir los parametros del entrenamiento.

```
from transformers import TrainingArguments

training_args = TrainingArguments(
```

```
    output_dir="detr_finetuned_kangaroo", # para guardar el modelo
    num_train_epochs=5, # número de épocas de entrenamiento
    fp16=True, # si usamos una precisión de fp16
    per_device_train_batch_size=8, # tamaño del batch
    dataloader_num_workers=2, # lo he ejecutado con 4 y me ha recomendado usar solo 2
    learning_rate=5e-5, # learning rate

    # otros parámetros recomendados
    lr_scheduler_type="cosine",
    weight_decay=1e-4,
    max_grad_norm=0.01,
    greater_is_better=True,
    load_best_model_at_end=True,
    eval_strategy="epoch",
    save_strategy="epoch",
    save_total_limit=2,
    remove_unused_columns=False,
    eval_do_concat_batches=False,

    # sobre huggingface: no lo subas a ningún lado
    push_to_hub=False,
    report_to="none"
)
```

```
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
```

Entrenamiento (entrenamiento =)

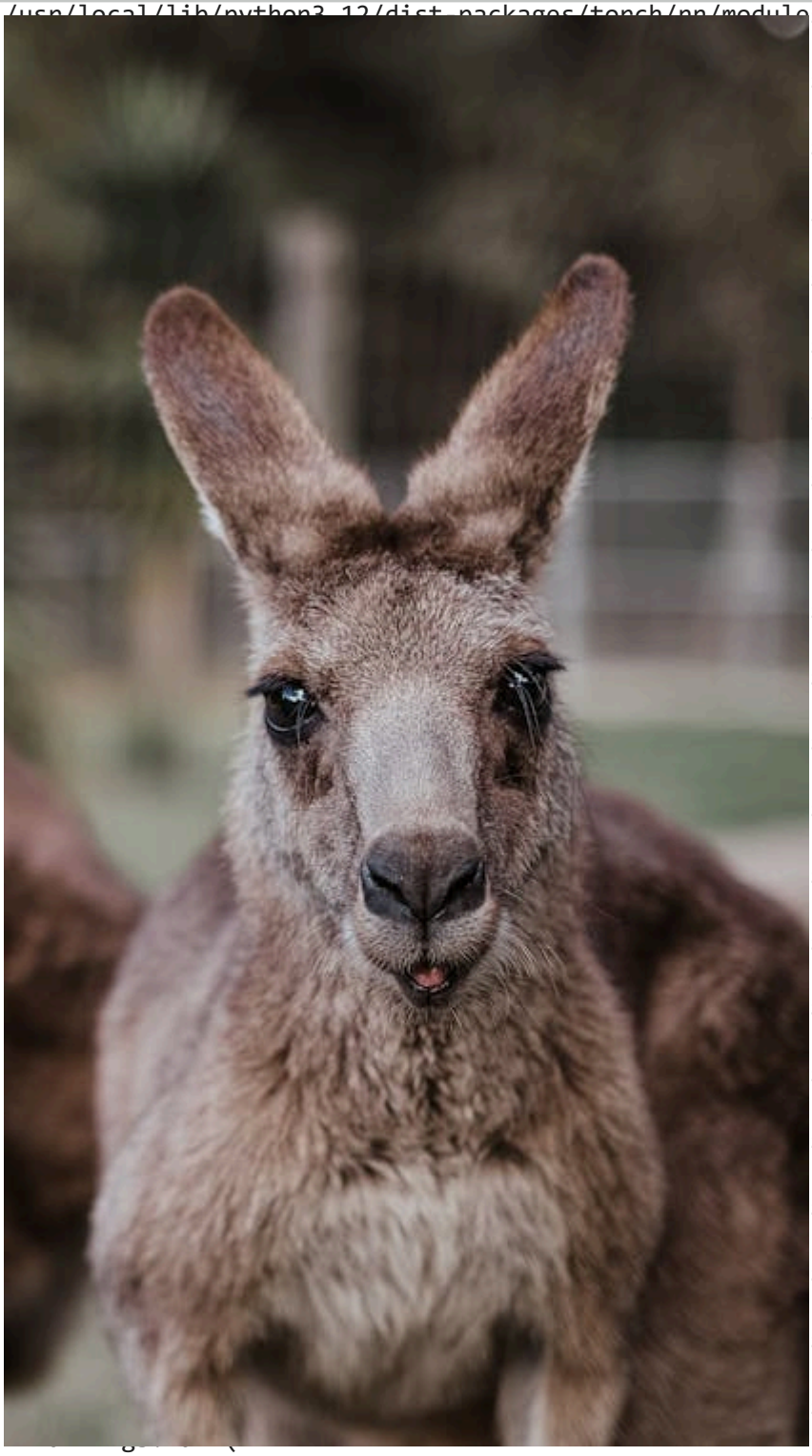
Y, finalmente...

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],
    processing_class=image_processor,
    data_collator=collate_fn
)

trainer.train()
```

```
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
```

```
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
```

```
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
```



```
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
```

```
for score, label, box in zip(results["scores"], results["labels"], results["boxes"]):
    box = [round(i, 2) for i in box.tolist()]
    print(
        f"Detected {model.config.id2label[label.item()]} with confidence "
        f"{round(score.item(), 3)} at location {box}"
    )
```

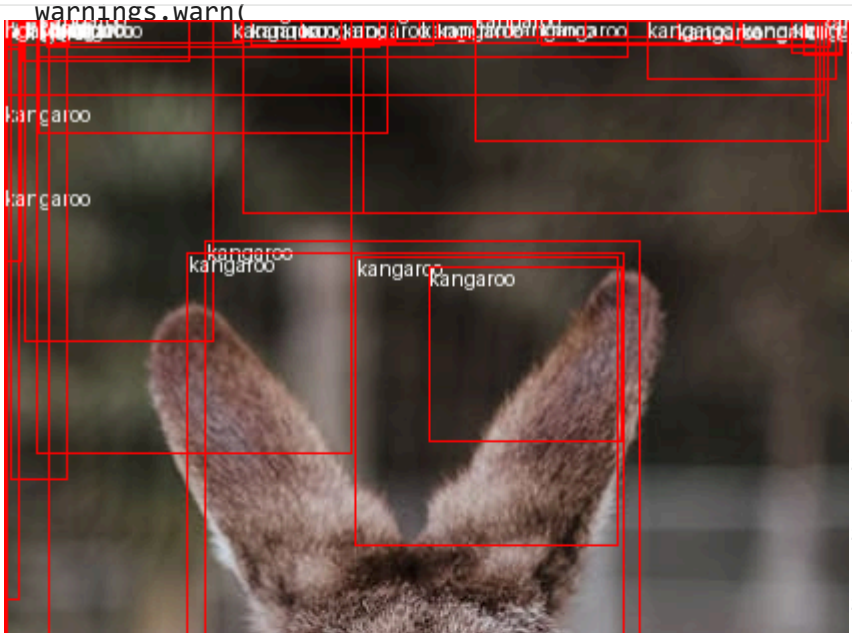
```
  warnings.warn(
Detected kangaroo with confidence 0.415 at location [-0.37, -2.52, 3.3, 20.88]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.415 at location [-0.08, -2.24, 1.13, 88.3]
  warnings.warn(
Detected kangaroo with confidence 0.413 at location [331.75, 730.55, 411.29, 749.68]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.412 at location [274.77, 727.19, 402.73, 749.38]
  warnings.warn(
Detected kangaroo with confidence 0.408 at location [-0.37, 636.61, 12.57, 751.33]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.407 at location [84.38, 724.69, 403.56, 749.68]
  warnings.warn(
Detected kangaroo with confidence 0.407 at location [66.56, 736.05, 394.18, 750.03]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.406 at location [398.42, 718.64, 421.68, 749.86]
  warnings.warn(
Detected kangaroo with confidence 0.406 at location [0.35, -1.15, 10.68, 14.07]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.403 at location [208.22, 735.59, 390.99, 749.77]
  warnings.warn(
Detected kangaroo with confidence 0.403 at location [102.62, 722.45, 403.46, 749.71]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.401 at location [17.41, 726.69, 128.06, 749.87]
  warnings.warn(
Detected kangaroo with confidence 0.397 at location [6.72, 677.63, 96.41, 749.13]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.395 at location [2.46, -0.4, 32.08, 12.78]
  warnings.warn(
Detected kangaroo with confidence 0.395 at location [300.84, 585.21, 411.29, 745.61]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.393 at location [-0.14, -12.27, 8.52, 120.21]
  warnings.warn(
Detected kangaroo with confidence 0.393 at location [-0.58, 559.52, 40.45, 745.65]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.392 at location [7.73, 665.59, 72.33, 748.72]
  warnings.warn(
Detected kangaroo with confidence 0.391 at location [60.48, 501.62, 416.75, 744.96]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.39 at location [32.56, 736.16, 212.85, 749.92]
  warnings.warn(
Detected kangaroo with confidence 0.39 at location [227.54, 613.72, 409.59, 748.58]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.389 at location [50.17, 737.67, 262.2, 749.88]
  warnings.warn(
Detected kangaroo with confidence 0.388 at location [9.76, -1.22, 92.39, 20.31]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.388 at location [67.69, 422.42, 414.89, 742.13]
  warnings.warn(
Detected kangaroo with confidence 0.388 at location [287.34, 414.0, 412.83, 741.76]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.387 at location [-0.1, 41.2, 1.9, 371.15]
  warnings.warn(
Detected kangaroo with confidence 0.386 at location [368.05, -0.58, 412.07, 13.1]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.385 at location [-0.57, 436.25, 9.52, 729.03]
  warnings.warn(
Detected kangaroo with confidence 0.385 at location [49.25, 363.1, 412.73, 747.25]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.384 at location [4.74, 738.22, 409.84, 750.06]
  warnings.warn(
Detected kangaroo with confidence 0.383 at location [383.27, 568.77, 420.75, 745.19]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.382 at location [18.96, 627.14, 420.35, 750.55]
  warnings.warn(
Detected kangaroo with confidence 0.381 at location [49.04, 381.13, 415.82, 746.06]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.38 at location [-0.2, -13.18, 7.52, 289.67]
  warnings.warn(
Detected kangaroo with confidence 0.38 at location [207.65, -1.09, 364.77, 11.6]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.379 at location [393.53, -0.63, 417.09, 16.31]
  warnings.warn(
Detected kangaroo with confidence 0.378 at location [-6.49, 734.8, 414.89, 750.56]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.378 at location [123.27, -0.9, 214.66, 12.13]
  warnings.warn(
Detected kangaroo with confidence 0.378 at location [-0.11, 82.76, 1.99, 375.0]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.377 at location [-0.31, 434.69, 67.38, 740.11]
  warnings.warn(
Detected kangaroo with confidence 0.377 at location [21.94, 687.26, 265.3, 748.28]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.376 at location [69.03, 652.4, 332.78, 749.45]
  warnings.warn(
Detected kangaroo with confidence 0.376 at location [148.41, -0.67, 234.23, 9.64]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.376 at location [217.28, 418.49, 410.77, 739.47]
  warnings.warn(
Detected kangaroo with confidence 0.376 at location [25.68, -0.78, 118.97, 11.8]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.374 at location [120.44, 395.42, 413.65, 742.03]
  warnings.warn(
Detected kangaroo with confidence 0.374 at location [21.79, -1.19, 187.05, 13.24]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.373 at location [343.44, 424.8, 418.55, 740.48]
  warnings.warn(
Detected kangaroo with confidence 0.373 at location [268.13, -0.91, 399.49, 12.32]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.373 at location [1.08, 391.48, 159.23, 740.11]
  warnings.warn(
Detected kangaroo with confidence 0.373 at location [2.47, 493.84, 119.72, 742.49]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.372 at location [58.0, 340.94, 419.22, 745.97]
  warnings.warn(
Detected kangaroo with confidence 0.372 at location [260.3, 456.12, 414.74, 740.66]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
Detected kangaroo with confidence 0.372 at location [252.37, -0.55, 367.03, 8.15]
  warnings.warn(
Detected kangaroo with confidence 0.372 at location [-5.98, 651.17, 415.82, 749.22]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
```

```
Detected kangaroo with confidence 0.372 at location [10.46, -21.19, 104.42, 160.45]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
Detected kangaroo with confidence 0.371 at location [6.49, 396.88, 424.16, 746.43]
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2446: UserWarning:
  warnings.warn(
```

Y también los podemos poner "sobre" la imagen

```
draw = ImageDraw.Draw(image)

for score, label, box in zip(results["scores"], results["labels"], results["boxes"]):
    box = [round(i, 2) for i in box.tolist()]
    x, y, x2, y2 = tuple(box)
    draw.rectangle((x, y, x2, y2), outline="red", width=1)
    draw.text((x, y), model.config.id2label[label.item()], fill="white")

image
```

```
  warnings.warn(
```




```
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
s/module.py:2446: UserWarning:
```